

Ch3 数据和C

对应《C Primer Plus（第6版）》第3章 · 本章目标：30 分钟读懂 + 60 分钟练完 · 不覆盖：位运算技巧、进制转换细节

3.1 数据在电脑里是怎么存的

计算机的"最小单位"是**位 (bit)**，只能存 0 或 1。8 个位组成一个**字节 (byte)**。一个 `int` 通常占 **4 字节 = 32 位**。你可以把变量理解成"一个有固定大小的盒子"——盒子多大（字节数），决定了它能装多大/多精的值。

一句话理解：类型决定两件事——**占几个字节**（能存多大）和**怎么解释这些字节**（整数还是小数还是字符）。这就是为什么"类型不匹配"会出乱码。

3.2 整数类型：int 及其亲戚

类型	通常占用	范围（示例）
short	2 字节	约 -32768 ~ 32767
int	4 字节	约 -21亿 ~ 21亿
long long	8 字节	约 $\pm 9.2 \times 10^{18}$

新手坑 1：int 溢出（最常见）。盒子的容量是固定的，放进去的值超出范围就会"溢出"——结果像一个坏掉的里程表那样**绕回**。C 语言**不报错**，这是最危险的地方。

```
/* ch3_01_overflow.c - int 溢出现象 */
#include <stdio.h>

int main(void)
{
    int big = 2147483647;    /* int 能装的最大值 */
    printf("big = %d\n", big);
    printf("big + 1 = %d\n", big + 1);    /* 溢出！会绕回成负数 */
    return 0;
}
```

运行输出（本机 gcc 16.1.0 实测）：big = 2147483647 big + 1 = -2147483648

为什么是 -2147483648? 因为 `int` 用最高位表示正负（补码）。最大值再加 1，二进制全 1 进位翻到符号位，就成了最小的负数。记住现象即可：**超出范围会绕回，编译器不提醒，只能靠判断范围。**

3.3 浮点类型：float 和 double

存小数用浮点：`float`（4 字节，精度约 7 位）和 `double`（8 字节，精度约 15 位）。日常运算默认用 **double**，除非明确要求省内存。

```
/* ch3_02_float.c - 浮点精度 */
#include <stdio.h>

int main(void)
{
    float f = 0.1f;
    double d = 0.1;
    printf("float 0.1  → %.10f\n", f);
    printf("double 0.1 → %.10f\n", d);
    return 0;
}
```

运行输出（实测）：float 0.1 → 0.1000000015 double 0.1 → 0.1000000000

新手坑 2：小数不"精确"。 0.1 在二进制里是无限循环小数，存不下。`float` 会把误差放大——看到 0.1000000015 别惊讶。**结论：不要用 == 判断两个浮点数相等**（见练习册）；金额、分数这类需要精确的，要么用整数（分），要么用高精度库。

3.4 字符类型：char 和 ASCII

`char` 存一个字符，其实是存这个字符的 **ASCII 编码数字**。'A' 在内存里就是数字 65。

```
/* ch3_03_char.c - char 就是数字 */
#include <stdio.h>

int main(void)
{
    char c = 'A';
    printf("字符: %c\n", c);
    printf("它的 ASCII 数字: %d\n", c);    /* %c 当字符打, %d 当数字打 */
    return 0;
}
```

运行输出（实测）：字符: A 它的 ASCII 数字: 65

伏笔：因为字符就是数字，所以 `c + 1` 会得到 66（即 'B'）。这在后面做字母大小写转换时会用到。

3.5 常量：不打算改的值

写死在代码里的字面量（42、3.14、'A'）是**字面常量**。想给常量起个名字、防止手滑改掉，用 `const`：

```
/* ch3_04_const.c - const 常量 */
#include <stdio.h>

int main(void)
{
    const double PI = 3.14159;
    int radius = 10;
    printf("圆面积 = %f\n", PI * radius * radius);
    return 0;
}
```

运行输出（实测）：圆面积 = 314.159000

新手坑 3： `PI = 3.0;` 这种对 `const` 变量的赋值，编译器会报错——这正是 `const` 想保护你的。别把 `const` 常量当变量改。

3.6 printf/scanf：%占位符要对上类型

上一章学过 `printf/scanf`，这一章重点是**类型必须对号入座**：

占位符	配什么类型	说明
<code>%d</code>	<code>int</code>	整数
<code>%ld</code>	<code>long</code>	长整数
<code>%f</code>	<code>float</code> （ <code>printf</code> 里也接受 <code>double</code> ）	小数，6 位
<code>%lf</code>	<code>double</code>	scanf 读 double 必须 %lf
<code>%c</code>	<code>char</code>	字符
<code>%s</code>	<code>char[]</code>	字符串

```
/* ch3_05_scanf.c - scanf 读 double 的坑 */
#include <stdio.h>
```

```
int main(void)
{
    double height;
    printf("输入身高: ");
    scanf("%lf", &height); /* double 必须 %lf, 且要 & */
    printf("身高 = %f\n", height);
    return 0;
}
```

新手坑 4: scanf 用 %f 读 double。这是 Ch4 已埋的坑，再强调一次：printf 里 %f 能打 double（自动转），但 scanf 里**必须** %lf 配 double、%f 配 float——写反了读出来的值是乱码。

3.7 新手坑集中营（本章最容易错的 4 件事）

1. **int 溢出不报错**（见 3.2）——算大数前先估算范围。
2. **浮点数 == 判断相等**（见 3.3）——用 $\text{fabs}(a-b) < 1e-6$ 这类容差判断。
3. **scanf 占位符和类型对不上**（见 3.6）——%d/%f/%lf 记牢。
4. **给 %d 传 double**：printf 里 printf("%d", 3.14) 输出是乱码（类型和格式对不上，gcc 开 -Wall 会警告）。

3.8 一分钟自测（答案见练习册）

判断对错：

- ① `int n = 2147483647; n + 1` 的结果一定是正数。
- ② `scanf("%lf", &x)` 可以读 double 类型。
- ③ 两个 double 用 `==` 判断相等是可靠的。

3.9 练习指引

做完本章内容后，去练习册完成 Ch3 配套练习；同时打开《C Primer Plus（第6版）》第 3 章，完成**复习题 1、3、5**（题号对应主教材）。改错题是练习册里最有价值的部分——先自己找 bug，再看答案的思路讲解。

本讲义为自写原创讲解，仅作课堂伴读；示例代码均可用 `gcc -Wall -Wextra` 编译，讲义每句「输出：」均已实跑核对。原书题目请以正版教材为准。